

```

#include <stdio.h> #include <stdbool.h> #include <SDL2/SDL.h>

#define TITLE      "game" #define WINDOW_WIDTH    800 #define WINDOW_HEIGHT    600 #define FPS
60

#define PLAYER_SPEED    4.0f #define SPRINT_MULTIPLIER 2.0f

#define PLAYER_WIDTH    48 #define PLAYER_HEIGHT    48

#define GRAVITY        0.5f #define JUMP_FORCE    -11.0f #define MAX_FALL_SPEED    12.0f

SDL_Window *window = NULL; SDL_Renderer *renderer = NULL;

struct player {

    float x, y, vx, vy;
    bool moving_left, moving_right;
    bool sprinting, grounded;
    int facing_dir;

} player;

int shouldQuit = 0;

float groundY = WINDOW_HEIGHT - PLAYER_HEIGHT;

void initSDL(){

    SDL_Init(SDL_INIT_EVERYTHING);

    window = SDL_CreateWindow(
        TITLE,
        0, //SDL_WINDOWPOS_CENTERED,
        0, //SDL_WINDOWPOS_CENTERED,
        WINDOW_WIDTH,
        WINDOW_HEIGHT,
        SDL_WINDOW_SHOWN
    );

    renderer = SDL_CreateRenderer(window, -1, 0);

}

void cleanup(){

    SDL_DestroyRenderer(renderer);
    SDL_DestroyWindow(window);
    SDL_Quit();

}

void setupLevel(){
    // TODO: add tilemap?

    player.x = (WINDOW_WIDTH / 2) - PLAYER_WIDTH;
    player.y = groundY;
    player.vx = 0;
    player.vy = 0;
    player.grounded = true;

}

```

```

void processInput(){
    // TODO: mouse movement?

    SDL_Event event;
    while (SDL_PollEvent(&event)){
        switch (event.type) {
            case SDL_QUIT:
                shouldQuit = 1;
                break;
            case SDL_KEYUP:
                if (event.key.keysym.sym == SDLK_ESCAPE)
                    shouldQuit = 1;
                if (event.key.keysym.sym == SDLK_q)
                    shouldQuit = 1;
                if (event.key.keysym.sym == SDLK_SPACE) {
                    if (player.vy < -4) {
                        player.vy = -4;
                    }
                }
                break;
            case SDL_KEYDOWN:
                if (event.key.repeat == 0) {

                    if (event.key.keysym.sym == SDLK_SPACE && player.grounded) {
                        player.grounded = false;
                        player.vy = JUMP_FORCE;
                    }
                }
                break;
        }
    }
    const Uint8 *state = SDL_GetKeyboardState(NULL);

    player.moving_left = state[SDL_SCANCODE_A];
    player.moving_right = state[SDL_SCANCODE_D];
    player.sprinting = state[SDL_SCANCODE_LSHIFT]; }

```

```

void update(){
    // TODO: collision detection, on other objects
    // TODO: collision on tilemap?
    // TODO: gravity
    // TODO: debug text with player coords

```

```

//player movement

```

```

player.vx = 0;

```

```

if (player.moving_left) {
    player.facing_dir = 0;
    if (player.sprinting) {
        player.vx = -PLAYER_SPEED * SPRINT_MULTIPLIER;
    } else
        player.vx = -PLAYER_SPEED;
}
if (player.moving_right) {
    player.facing_dir = 1;
    if (player.sprinting) {
        player.vx = +PLAYER_SPEED * SPRINT_MULTIPLIER;
    } else
        player.vx = +PLAYER_SPEED;
}

```

```

player.vv += GRAVITY;

```

```

if (player.vy > MAX_FALL_SPEED) {
    player.vy = MAX_FALL_SPEED;
}

player.x += player.vx;
player.y += player.vy;

//border collision

if (player.y <= 0) {
    player.y = 0;
    player.vy = 0;
}

if (player.y >= groundY) {
    player.y = groundY;
    player.grounded = true;
    player.vy = 0;
} else {
    player.grounded = false;
}

if (player.x <= 0) {
    player.x = 0;
}
if (player.x >= WINDOW_WIDTH - PLAYER_WIDTH) {
    player.x = WINDOW_WIDTH - PLAYER_WIDTH;
}

SDL_Delay(1000 / FPS); }

void render(){
    // TODO: render player sprite
    // TODO: render tile map
    // TODO: render debug text

    SDL_SetRenderDrawColor(renderer, 0, 0, 0, 255);
    SDL_RenderClear(renderer);

    SDL_Rect player_rect = {
        player.x,
        player.y,
        PLAYER_WIDTH,
        PLAYER_HEIGHT
    };

    SDL_SetRenderDrawColor(renderer, 255, 255, 255, 255);
    SDL_RenderFillRect(renderer, &player_rect);

    SDL_RenderPresent(renderer);
}

int main(){

    initSDL();
    setupLevel();

    while (shouldQuit != 1){

        processInput();
        update();
        render();
    }
}

```

```
}
```

```
cleanup();  
printf("All good!\n");  
return 0; }
```